

UD 4 Realización de consultas

SELECT

Cláusula SQL	Función	Obligatorio
SELECT	Muestra una lista de los campos que contienen datos de interés.	Sí
FROM	Muestra las tablas que contienen los campos de la cláusula SELECT.	Sí
WHERE	Especifica los criterios de campo que cada registro debe cumplir para poder ser incluido en los resultados.	No
ORDER BY	Especifica la forma de ordenar los resultados.	No
GROUP BY	En una instrucción SQL que contiene funciones de agregado, muestra los campos que no se resumen en la cláusula SELECT.	Solo si están estos campos
HAVING	En una instrucción SQL que contiene funciones de agregado, especifica las condiciones que se aplican a los campos que se resumen en la instrucción SELECT.	No

SINTAXIS

```
SELECT field_1  
FROM table_1  
WHERE criterion_1
```



```
SELECT NOMBRE, CIUDAD  
FROM CLIENTES  
WHERE CIUDAD="MALAGA";
```



cliente Málaga	
NOMBRE	CIUDAD
ALBERTO GONZALEZ	MALAGA
MARIO VARGAS	MALAGA
ALFREDO SANEZ	MALAGA
BARTOLOME SANCHEZ	MALAGA
CARMEN RUIZ perez	MALAGA
MARIA RUIZ	MALAGA
MARIO SANCHEZ	MALAGA
CORSAN	MALAGA
CORTES INGLES	MALAGA
ENDESA	MALAGA
MICROSOFT IBERICA	MALAGA

OPERADORES ARITMÉTICOS

Tenemos los siguientes operadores y su operación:

Operadores y su significado.	
OPERADOR	SIGNIFICADO
=	Igualdad.
!=, < >, ^=	Desigualdad.
<	<
>	Mayor que.
<=	Menor o igual que.
>=	Mayor o igual que.
IN	Igual que cualquiera de los miembros entre paréntesis.
NOT IN	Distinto que cualquiera de los miembros entre paréntesis.
BETWEEN	Entre. Contenido dentro del rango.
NOT BETWEEN	Fuera del rango.
LIKE '_abc%'	Se utiliza sobre todo con textos y permite obtener columnas cuyo valor en un
	campo cumpla una condición textual. Utiliza una cadena que puede contener los
	símbolos "%" que sustituye a un conjunto de caracteres o "_" que sustituye a un
	carácter.
IS NULL	Devuelve verdadero si el valor del campo de la fila que examina es nulo.

OTROS OPERADORES

Operadores aritméticos y su significado.

OPERADOR	SIGNIFICADO
+	Suma
-	Resta
*	Multiplicación
/	División

Operadores lógicos y su significado.

OPERADOR	SIGNIFICADO
AND	Devuelve verdadero si sus expresiones a derecha e izquierda son ambas verdaderas.
OR	Devuelve verdadero si alguna de sus expresiones a derecha o izquierda son verdaderas.
NOT	Invierte la lógica de la expresión que le precede, si la expresión es verdadera devuelve falsa y si es falsa devuelve verdadera.

Consultas campos calculadas

- Estos **campos calculados** se obtienen a través de la sentencia `SELECT` poniendo a continuación la expresión que queramos. Esta consulta **no modificará los valores originales** de las columnas ni de la tabla de la que se está obteniendo dicha consulta, únicamente mostrará una columna nueva con los valores calculados.
- Por ejemplo: `SELECT Nombre, Credito, Credito + 25 FROM USUARIOS;` Con esta
- consulta hemos creado un campo que tendrá como nombre la expresión utilizada. Podemos ponerle un alias a la columna creada añadiéndolo detrás de la expresión junto con la palabra `AS`. En nuestro ejemplo quedaría de la siguiente forma:
- `SELECT Nombre, Credito, Credito + 25 AS CreditoNuevo FROM USUARIOS;`
- `SELECT precio, precio * 0.21 AS iva, precio + (precio * 0.21) AS total FROM articulos`

JOIN

INNER JOIN

TODOS LOS DATOS QUE CORRESPONDAN ENTRE UNA TABLA Y OTRA

```
SELECT * FROM clientes INNER JOIN ventas  
ON clientes.idclientes = ventas.idclientes
```

LEFT JOIN

TODOS LOS DATOS DE LA TABLA IZQUIERDA Y SOLO LOS DE LA TABLA DERECHA QUE SEAN IGUALES

```
SELECT * FROM clientes LEFT JOIN ventas  
ON clientes.idclientes = ventas.idclientes
```

RIGHT JOIN

TODOS LOS DATOS DE LA TABLA DERECHA Y SOLO LOS DE LA TABLA IZQUIERDA QUE SEAN IGUALES

```
SELECT * FROM clientes RIGHT JOIN ventas  
ON clientes.idclientes = ventas.idclientes
```

OUTER JOIN

ES UNA COMBINACION DE LEFT JOIN Y RIGHT JOIN, EJEMPLO LEFT OUTER JOIN, RIGHT OUTER JOIN, en el ejemplo nos mostraria todos los clientes aunque no tengan correlación con la tabla ventas

```
SELECT * FROM clientes OUTER JOIN  
ventas ON clientes.idclientes =  
ventas.idclientes
```

NATURAL JOIN

RELACIONA TABLAS QUE TENGAN EL MISMO NOMBRE DE COLUMNA

```
SELECT * FROM clientes NATURAL JOIN ventas
```

JOIN USING

INDICAMOS SOLO LA TABLA QUE ESTA RELACIONADA Y POR EL CAMPO QUE ESTA RELACIONADO

```
SELECT * FROM clientes JOIN ventas USING  
(idclientes)
```

JOIN ON

La cláusula JOIN ON es similar a INNER JOIN, pero permite especificar la condición de unión en lugar de utilizar el nombre de la columna para unir las tablas

```
SELECT * FROM clientes JOIN ventas  
ON clientes.idclientes =  
ventas.idclientes
```

FUNCIONES INTEGRADAS DE SQL

Función	Parámetros	Valor devuelto
ABS	Número	Valor absoluto del número
MOD	Número 1, Número 2	Módulo (resto) de la división del primer número entre el segundo
POSITION	Cadena de caracteres 1, Cadena de caracteres 2	Posición de la primera cadena dentro de la segunda
CHAR_LENGTH	Cadena de caracteres	Tamaño de la cadena
SUBSTRING	Cadena de caracteres, Posición de inicio, Número de caracteres	Cadena de caracteres dentro de la cadena recibida como parámetro. El segundo parámetro define la posición de la cadena original a partir de la que se extraerá la cadena que se va a devolver, y el tercero el número de caracteres de la cadena resultante
UPPER	Cadena de caracteres	Cadena de caracteres original en mayúsculas
LOWER	Cadena de caracteres	Cadena de caracteres original en minúsculas

```
SELECT ABS(SUM(nImporte))  
FROM TFactura  
WHERE dFactura BETWEEN "2013-01-01" AND "2013-12-31";
```

Ejercicios

- Ejemplo 1:
- `SELECT ABS(SUM(articulos.PRECIO)) FROM articulos WHERE articulos.FABRICANTE="APPLE"`
- `SELECT fabricante, POSITION("A" in fabricante) FROM articulos`
- `SELECT fabricante, CHAR_LENGTH(fabricante) FROM articulos`
- `SELECT fabricante, substring(fabricante,1,2) FROM articulos`
- `SELECT lower(fabricante) FROM articulos`

Ejemplo de funciones en Mysql

- Sum

```
SELECT SUM(precio) FROM artículos
```

- YEAR, MONTH, DAY

```
SELECT year(persona.fecha_nacimiento) FROM persona
```

- DATE_FORMAT

- ```
SELECT persona.fecha_nacimiento AS 'Fecha nacventa',
```
- ```
DATE_FORMAT(persona.fecha_nacimiento, '%d %M %Y')
```
- ```
AS 'Nuevo formato'
```
- ```
FROM persona;
```

Ejemplo de funciones en Mysql

- DATEDIFF

- `SELECT persona.fecha_nacimiento AS 'Fecha venta',`
- `FORMAT((DATEDIFF(NOW(), persona.fecha_nacimiento))/365,0)`
- `AS 'Antigüedad en días' FROM persona;`

- REPLACE

```
SELECT TIPO, REPLACE(TIPO, 'alumno', 'pupilo') AS 'tipo reemplazado' FROM
personapersona
```

- Avg

```
SELECT avg(articulos.PRECIO) FROM articulos
```

- Concat

```
SELECT CONCAT(articulos.id_PRODUCTO, SUBSTRING(fabricante,1,2)) AS CODIGOFINAL FROM
articulos
```

Subconsultas

- El estándar SQL define tres tipos de subconsultas:
- Subconsultas de fila. Son aquellas que devuelven más de una columna pero una única fila.
- Subconsultas de tabla. Son aquellas que devuelve una o varias columnas y cero o varias filas.
- Subconsultas escalares. Son aquellas que devuelven una columna y una fila.

Subconsultas en la cláusula WHERE

Subconsultas en la cláusula HAVING

Subconsultas en la cláusula FROM

Subconsultas en la cláusula SELECT

Operadores que podemos usar en las subconsultas

Tipos de subconsultas

cláusula
WHERE



```
SELECT nombre  
FROM artículos  
WHERE precio = (SELECT MAX(precio) FROM artículos);
```

cláusula
HAVING



```
SELECT sum(unidades), fabricante, precio,  
COUNT(nombre)  
FROM artículos INNER JOIN ventas ON  
ventas.id_PRODUCTOS=artículos.id_PRODUCTO  
GROUP BY fabricante having count(nombre)=14  
ORDER BY fabricante
```

cláusula
HAVING

```
SELECT unidades, fabricante, precio,  
nombre, FORMAT((SELECT AVG(precio) FROM  
artículos), 2)  
FROM artículos INNER JOIN ventas ON  
ventas.id_PRODUCTOS=artículos.id_PRODUCTO  
WHERE precio > (SELECT AVG(precio) FROM  
artículos)
```

OPERADORES

Los operadores que podemos usar en las subconsultas son los siguientes:

Operadores básicos de comparación (>, >=, <, <=, !=, <>, =).

Predicados ALL y ANY .

Predicado IN y NOT IN .

Predicado EXISTS y NOT EXISTS

Subconsultas con ALL y ANY

- ALL y ANY se utilizan con los operadores de comparación (>, >=, <, <=, !=, <>, =) y nos permiten comparar
- una expresión con el conjunto de valores que devuelve una subconsulta.
- ALL y ANY los vamos a utilizar para realizar comparaciones con subconsultas que pueden devolver varios
- valores, es decir, una columna y varias filas.

EN MYSQL USAMOS ANY O SOME

SELECT

articulos.fabricante, COUNT(nombre), AVG(precio)

FROM

articulos

GROUP BY

articulos.fabricante

HAVING

COUNT(nombre) >= 2 AND AVG(precio) > ANY (SELECT AVG(precio) FROM articulos);

Subconsultas con IN y NOT IN

IN y NOT IN nos permiten comprobar si un valor está o no incluido en un conjunto de valores, que puede ser el conjunto de valores que devuelve una subconsulta.

IN y NOT IN los vamos a utilizar para realizar comparaciones con subconsultas que pueden devolver varios valores, es decir, una columna y varias filas.

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
SELECT *  
FROM clientes  
WHERE id_clientes NOT IN (SELECT id_clientes FROM  
ventas);
```

Subconsultas con EXISTS y NOT EXISTS

Devuelve un listado de los clientes que no han realizado ningún pedido.

- **SELECT** * **FROM** clientes
- **WHERE NOT EXISTS** (SELECT ventas.Id_CLIENTES FROM ventas
- **WHERE** clientes.id_clientes = ventas.id_clientes);

Union intersección y diferencia de consultas

UNION



SOBRE DOS CONSULTAS, DEVUELVE LAS FILAS DE LA PRIMERA CONSULTA MÁS LOS DE LA SEGUNDA ELIMINADO FILAS REPETIDAS

SELECT ID_PEDIDOS FROM PEDIDO UNION SELECT ID_PEDIDOS FROM PEDIDOS2

INTERSECT



NO EN MYSQL

SOBRE DOS CONSULTAS, DEVUELVE LAS FILAS IGUALES EN AMBAS CONSULTAS

SELECT ID_PEDIDOS FROM PEDIDO INTERSECT SELECT ID_PEDIDOS FROM PEDIDOS2

MINUS



NO EN MYSQL

SOBRE DOS CONSULTAS, DEVUELVE LAS FILAS DE LA PRIMERA CONSULTA, MENOS LAS DE LA SEGUNDA

SELECT ID_PEDIDOS FROM PEDIDO MINUS SELECT ID_PEDIDOS FROM PEDIDOS2

2.9 CONSULTAS DE TABLAS DE REFENCIAS CRUZADAS

NO EN MYSQL

Ejemplo

- Creamos un duplicado de la tabla ventas de phoneland

```
CREATE TABLE ventas1 LIKE VENTAS;  
INSERT INTO ventas1 SELECT * FROM ventas1;
```

- Entramos en ventas1 y quitamos los id de ventas del 1 al 60

```
DELETE FROM ventas1  
WHERE id_ventas <= 60;
```

- Lo que hacemos ahora es unir ambas tablas

```
SELECT id_ventas FROM ventas1  
UNION  
SELECT id_ventas FROM ventas  
ORDER BY id_ventas;
```

Consultas de agrupación

- **GROUP BY:** Agrupa los resultados de una consulta por una o varias columnas. Por ejemplo, si queremos obtener el total de ventas por departamento, podríamos utilizar la sentencia `SELECT departamento, SUM(ventas) FROM tabla GROUP BY departamento`.
- **HAVING:** Filtra los resultados de una consulta agrupada por una condición. Por ejemplo, si queremos obtener el total de ventas por departamento para aquellos departamentos que tienen más de 1000 ventas, podríamos utilizar la sentencia `SELECT departamento, SUM(ventas) FROM tabla GROUP BY departamento HAVING SUM(ventas) > 1000`.
- **WITH ROLLUP:** Agrega subtotales y totales a los resultados de una consulta agrupada. Por ejemplo, si queremos obtener el total de ventas por departamento y un total global, podríamos utilizar la sentencia `SELECT departamento, SUM(ventas) FROM tabla GROUP BY departamento WITH ROLLUP`.
- **WITH CUBE:** Agrega subtotales para todas las posibles combinaciones de columnas en los resultados de una consulta agrupada. Por ejemplo, si queremos obtener el total de ventas por departamento, por año y por mes, podríamos utilizar la sentencia `SELECT departamento, año, mes, SUM(ventas) FROM tabla GROUP BY departamento, año, mes WITH CUBE`. MariaDb no soporta with Cube.

Ejemplo con WITH ROLLUP

SELECT

`articulos.fabricante, COUNT(nombre), AVG(precio)`

FROM

`articulos`

GROUP BY

`articulos.fabricante WITH ROLLUP`

HAVING

`COUNT(nombre) >= 2 AND AVG(precio) > ANY (SELECT AVG(precio) FROM articulos);`

- La opción "WITH ROLLUP" agrupa los resultados y agrega una fila adicional con el total de todos los grupos. En este caso, la cláusula "HAVING" se aplica a cada grupo individual y al total, lo que permite ver los resultados agregados.

FUNCIONES DE CONSULTAS AGRUPADAS

- MySQL proporciona varias funciones de agregado para realizar consultas agrupadas en una tabla. Las funciones de agregado más comunes son:
- **AVG():** devuelve el promedio de los valores seleccionados.
- **COUNT():** cuenta el número de filas seleccionadas.
- **MAX():** devuelve el valor máximo de los valores seleccionados.
- **MIN():** devuelve el valor mínimo de los valores seleccionados.
- **SUM():** devuelve la suma de los valores seleccionados.
- Para usar estas funciones, debes usar una cláusula GROUP BY en tu consulta para agrupar los resultados por una o más columnas. Por ejemplo:
- *SELECT COUNT(*), country*
- *FROM customers*
- *GROUP BY country;*
- Esta consulta cuenta el número de clientes por país y devuelve dos columnas: la columna COUNT(*) con el número de clientes por país y la columna country con el nombre del país.

Consultas de referencias cruzadas (ORACLE)

Para realizar una consulta de referencia cruzada en Oracle, puedes usar la función PIVOT. La sintaxis básica de la función PIVOT es la siguiente:

Copy code

```
SELECT *  
FROM (  
  SELECT mes, cliente, importe  
  FROM ventas  
)  
PIVOT (  
  SUM(importe)  
  FOR mes IN (enero, febrero, marzo, abril, mayo, junio, julio, agosto, septiembre, octubre, noviembre, diciembre)  
);
```

Esta consulta selecciona todas las ventas de la tabla ventas, agrupa los resultados por cliente y mes y suma los importes de las ventas por mes. La función PIVOT convierte los meses en columnas y muestra el importe total de las ventas para cada mes y cada cliente.

CONSULTAS DE INSERCIÓN

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

EJEMPLO: INSERT INTO PRODUCTOS (NOMBRE, FABRICANTE, PRECIO)
VALUES ("PORTATIL HP", "HP", 700)

SINTAXIS:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

EJEMPLO

```
UPDATE CLIENTES
SET NOMBRE = 'Alberto Gonzalez',
    Ciudad= 'Granada'
WHERE id_clientes = 1
```

SINTAXIS:

```
DELETE FROM table_name WHERE condition;
```

EJEMPLO

```
DELETE FROM clients Where id_cliente=1
```

CONSULTAS DE BORRADO

Ejemplos consultas agrupadas con campos calculados

```
SELECT FABRICANTE, Sum(PRECIO) AS  
SumaDePRECIO,  
Sum(PRECIO * 0.21) AS IVA, SUM(precio) +  
Sum(PRECIO * 0.21) AS Total  
FROM articulos  
GROUP BY FABRICANTE;
```

USANDO VARIABLES

```
SET @IVA = 0.21;  
SELECT FABRICANTE,  
        @SUMAPRECIO:=Sum(PRECIO) AS  
SumaDePRECIO,  
        @IVACAL:=Sum(PRECIO * @IVA) AS IVA,  
        @SUMAPRECIO + Sum(PRECIO * @IVA) AS  
Total  
FROM articulos  
GROUP BY FABRICANTE;
```

REPLACE Y SHOW

REPLACE

REPLACE ES IGUAL QUE INSERT PERO REEMPLAZA EN VEZ DE INSERTAR

SU SINTAXIS ES IGUAL A LA DE INSERT

MUESTRA INFORMACION DEL TIPO

SHOW CARÁCTER SET

SHOW COLLATION

SHOW ENGINES

SHOW CREATE DATABASE

SHOW TABLES- use pedidos show tables

SHOW CREATE TABLES-show create table pedidos

SHOW COLUMNS-COLUMNAS DE UNA TABLA

Show columns from cliente

SHOW

LENGUAJE DE CONTROL DCL

Control de acceso a los datos:

- En MySQL, puedes controlar el acceso a los datos mediante la creación de usuarios y asignación de permisos. Al crear un usuario, puedes especificar qué bases de datos y tablas pueden acceder y qué acciones pueden realizar sobre ellas.

CONCESION DE PRIVILEGIOS



Se realiza sobre una tabla, una base de datos o un objeto de la base de datos, los permisos Son de lectura, select. De inserción insert, o de actualizado update, o de borrado delete Se realiza mediante la opción WITH GRANT OPTION

```
GRANT [CONNECT|RESOURCE|DBA|ALL PRIVILEGES|SELECT|UPDATE|INSERT|DELETE]
ON <objeto>
TO <lista_usuarios>
[WITH GRANT OPTION];
```

```
GRANT ALL PRIVILEGES ON phoneland.* TO 'alberto'@'localhost';
```

CREACIÓN DE USUARIOS



```
CREATE USER 'alberto' 'localhost'
```

REVOCAR PERMISOS USUARIOS



```
REVOKE [CONNECT|RESOURCE|DBA|ALL PRIVILEGES|SELECT|UPDATE|INSERT|DELETE]
FROM <lista_usuarios>
[ON <lista_objetos>];
```

```
REVOKE ALL PRIVILEGES ON phoneland.* FROM 'alberto'@'localhost';
```

Revocar permisos solo seleccionar, insertar

```
REVOKE SELECT, INSERT ON phoneland.* FROM 'alberto'@'localhost';
```

Create user

Especificación de restricciones de integridad

Instrucciones de autorización



CREATE USER- para crear usuarios sintaxis CREATE USER usuario [IDENTIFIED BY [PASSWORD] 'contraseña'

- 1.CREATE USER 'usuario'@'localhost' IDENTIFIED BY 'password';
- 2.GRANT ALL PRIVILEGES ON *. * TO 'nombre_usuario'@'localhost';
- 3.SELECT User FROM **mysql**. user;
- 4.DROP USER 'usuario'@'localhost';
- 5.REVOKE UPDATE, DELETE ON *. * FROM 'usuario'@'localhost';
- 6.REVOKE ALL PRIVILEGES ON *. * FROM 'usuario'@'localhost';

Transacciones

TRANSACCIONES Y PROPIEDADES

Una transacción es un conjunto de instrucciones escritas en un lenguaje de control o manipulación de datos

Atomicidad

(si las transacciones se realizan adecuadamente)

Consistencia

(la ejecución conserve la consistencia)

Aislamiento (si se ejecutan varias transacciones, el sistema garantiza que cada transacción ignore el resto de transacciones)

Permanencia o durabilidad (los cambios permanecen aunque haya fallos en el sistema)

Estados de una transacción

- **Activa:** es el estado inicial, en el cual la transacción permanece durante su ejecución.
- **Parcialmente comprometida,** estado al que llega después de ejecutar la última instrucción.
- **Fallida,** después de descubrir que no puede continuar su ejecución normal.
- **Abortada,** después de haber retrocedido la transacción y restablecido la base de datos a un estado anterior al comienzo de la transacción.
- **Comprometida,** tras completarse con éxito.

El diagrama de estados correspondiente a una transacción se muestra en la siguiente figura:

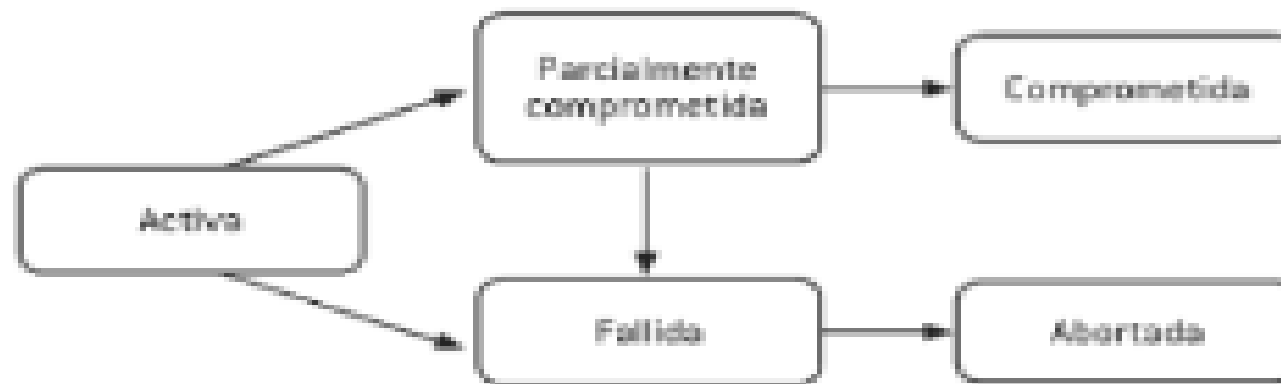


Figura 2.18. Diagrama de transición de estados de una transacción.

CONTIENE TODA LA INFORMACION SOBRE LOS DATOS
ALMACENADOS EN LA BASE DE DATOS



SE ENCUENTRE EN INFORMATION_SCHEMA, SE COMPONE
DE UNA SERIE DE TABLAS:
SCHEMATA, TABLES, COLUMNS, TABLE CONSTRAINTS, KEY
COLUMN USAGE, VIEW, USER, DB, HOST, TABLES PRIV,
COLUMNS PRIV, PROCS_PRIV,

Procesamiento de una consulta

La manera en que interpreta y ejecuta el sistema,
Una consulta escrita en un lenguaje de manipulación
De datos.



Tareas en el procesamiento de una consulta



Análisis lexico
Análisis sintactico
validación

COMMIT WORK- TODAS LAS ACTUALIZACIONES
INSERCCIONES, BORRADO O ACTUALIZACIONES SI ESTA
ACTIVADO SE RELIZAN AUTOMATICAMENTE

ROLLBACK WORK- LA TRANSACCION ACTUAL ABORTA

OPTIMIZACIÓN de una consulta

Generar expresiones equivalente a la expresión dada
Mediante una serie de reglas de equivalencia
Escoger un plan de evaluación de consultas.

Herramientas de la base de datos para la optimización
de consultas

Explain



Con este comando obtenemos la información sobre como se llevaran
a cabo las consultas

